A4 Arrays Basics and Algorithmic Thinking

Modern Code
MC (MTD.ba VZ)
SS 2025

Submission via Moodle — **24 Points**

This assignment focuses on algorithmic thinking with one-dimensional arrays. You will implement various array algorithms that demonstrate problem-solving skills and understanding of array operations. These exercises go beyond the basic operations covered in class and require you to think algorithmically.

Submission Requirements:

- Read the `Organizational Guidelines on Moodle`, and adhere to them!
- Submit a PDF document containing your **solution idea, code, and tests for each exercise**
- Include all Java source files (.java) in a ZIP archive
- As tests include copies or screenshots of the console output
- Use the naming convention: `MC_UE04_LASTNAME.pdf` and `MC_UE04_LASTNAME.zip`

Important: This assignment emphasizes algorithmic thinking and problem-solving. For each exercise, think about:

- What is the problem asking me to do?
- What steps do I need to take to solve it?
- How can I use arrays effectively?
- What edge cases should I consider?

Hint: You can reuse methods from class exercises, such as methods for random array initialization and sorting algorithms. Don't reinvent the wheel - build on what you've already learned!

Exercise 1 (4 points) Array Statistics: Mean, Median, Min, Max

Create a method that calculates statistical measures of an array. The method does not need to return anything and can just display the results directly.

1. Create an array of integers (you can initialize it with values or use random values)
2. Calculate and display:
   - **Mean (Average):** Sum of all elements divided by the number of elements
   - **Median:** The middle value when the array is sorted. If the array has an even number of elements, take the average of the two middle values
   - **Minimum:** The smallest value in the array
   - **Maximum:** The largest value in the array
3. Test with arrays of different sizes (including odd and even lengths for median)
4. Use proper variable names and include comments

Expected Output:

```
=== Array Statistics ===
Array: [15, 8, 23, 4, 42, 11, 19]
Mean: 17.43
Median: 15.0
Minimum: 4
Maximum: 42
```

Exercise 2 (4 points) Array Reversal: In-Place Algorithm

Create a method that reverses an array **in-place** (without using a second array) and returns the reversed array.

1. Create an array of integers
2. Create a method that reverses the array in-place by swapping elements and returns the reversed array
3. Display the array before and after reversal
4. **Important:** You must NOT create a second array. All operations must be done on the original array.

Expected Output:

```
=== Array Reversal ===
Original array: [10, 20, 30, 40, 50]
Reversed array: [50, 40, 30, 20, 10]
```

Exercise 3 (4 points) Fuzzy Search: Find Value or Closest Value

Create a method that searches for a target value in an array. If the exact value is found, return its index. If not found, return the index of the closest value. The method should return the index (you may display output within the method).

1. Create an array of integers
2. Create a method that searches for a target value and returns the index (exact match or closest)
3. Display the search results
4. Test with values that exist and values that don't exist

Expected Output:

```
=== Fuzzy Search ===
Array: [15, 8, 23, 4, 42, 11, 19]
Searching for: 20
Exact match not found.
Closest value: 19 at index 6
Difference: 1

Searching for: 8
Found exact match at index 1
```

Exercise 4 (4 points) Merging Two Arrays

Create a method that merges two arrays into one and returns the merged array.

1. Create two arrays of integers
2. Create a method that merges them into a single array and returns the merged array
3. Display all three arrays (the two original arrays and the merged result)
4. Test with arrays of different sizes

Expected Output:

```
=== Merging Two Arrays ===
Array 1: [10, 20, 30]
Array 2: [40, 50, 60, 70]
Merged array: [10, 20, 30, 40, 50, 60, 70]
```

Exercise 5 (4 points) Unique Values and Frequency Counting

Create a method that finds all unique values in an array and counts how often each value occurs. The method does not need to return anything and can just display the results directly.

1. Create an array that may contain duplicate values
2. Find all unique values (like a set - each value appears only once in the result)
3. Count how many times each unique value appears in the original array
4. Display each unique value and its frequency

Expected Output:

```
=== Unique Values and Frequency ===
Original array: [5, 2, 8, 2, 5, 5, 1, 8]
Unique values and frequencies:
  5 appears 3 times
  2 appears 2 times
  8 appears 2 times
  1 appears 1 time
```

Exercise 6 (4 points) Array Splitting: Evens and Odds

Create a method that splits an array into two arrays: one containing all even numbers and one containing all odd numbers. The method does not need to return anything and can just display the results directly.

1. Create an array of integers
2. Split it into two arrays:
   - One array with all even numbers
   - One array with all odd numbers
3. Preserve the original order within each group (evens and odds)
4. Display the original array and both resulting arrays

Expected Output:

```
=== Array Splitting: Evens and Odds ===
Original array: [15, 8, 23, 4, 42, 11, 19]
Even numbers: [8, 4, 42]
Odd numbers: [15, 23, 11, 19]
```